

Assignment 1: Analysis of Algorithms & Searching and Sorting

Madeline LeBreton

T00785871

COMP 2231

Submitted 18/06/2025

Objective

The objective of this lab was to investigate the performance of the shellSort, bubbleSort, and bubbleSort2 sorting algorithms. The methods were modified to output the number of comparisons, swaps, and total time in order to analyze the efficiency and time complexity of each.

Data

The driver was written to execute six sets of tests for each of the three sorting algorithms, differing by the number of integers in the array (10, 100, or 1000), and whether the array was sorted or unsorted. The number of comparisons, swaps, and total time was recorded for each.

```
Original data for rand10: [285734, -76926, -810205, -119061, 403474, -9288, -788972, -787584, -898337, 801119]

bubbleSort:
Final sorted array: [-898337, -810205, -788972, -787584, -119061, -76926, -9288, 285734, 403474, 801119]
Comparisons: 45
Swaps: 25
Total time: 0.0318 ms

bubbleSort2:
Final sorted array: [-898337, -810205, -788972, -787584, -119061, -76926, -9288, 285734, 403474, 801119]
Comparisons: 81
Swaps: 25
Total time: 0.0145 ms

shellSort:
Final sorted array: [-898337, -810205, -788972, -787584, -119061, -76926, -9288, 285734, 403474, 801119]
Comparisons: 69
Swaps: 15
Total time: 0.0096 ms
```

Figure 1: Data output for an unsorted array of size 10

```
Original data for ordered10: [-723892, -464365, 202678, 617440, 771231, 792919, 926876, 928625, 980065, 998776]

bubbleSort:
Final sorted array: [-723892, -464365, 202678, 617440, 771231, 792919, 926876, 928625, 980065, 998776]
Comparisons: 45
Swaps: 0
Total time: 0.0020 ms

bubbleSort2:
Final sorted array: [-723892, -464365, 202678, 617440, 771231, 792919, 926876, 928625, 980065, 998776]
Comparisons: 9
Swaps: 0
Total time: 0.0007 ms

shellSort:
Final sorted array: [-723892, -464365, 202678, 617440, 771231, 792919, 926876, 928625, 980065, 998776]
Comparisons: 22
Swaps: 0
Total time: 0.0002 ms
```

Figure 2: Data output for a sorted array of size 10

```
Original data for rand100: [-100079, 996162, 273799, -754090, -68063, -704777, 374597, -865541, -961028, -577623, -309288, -235]

bubbleSort:
Final sorted array: [-986438, -961028, -949003, -938178, -909474, -903384, -887189, -882289, -865961, -865541, -861621, -827583]
Comparisons: 4950
Swaps: 2328
Total time: 0.2673 ms

bubbleSort2:
Final sorted array: [-986438, -961028, -949003, -938178, -909474, -903384, -887189, -882289, -865961, -865541, -861621, -827583]
Comparisons: 8514
Swaps: 2328
Total time: 0.3696 ms

shellSort:
Final sorted array: [-986438, -961028, -949003, -938178, -909474, -903384, -887189, -882289, -865961, -865541, -861621, -827583]
Comparisons: 3098
Swaps: 506
Total time: 0.0412 ms
```

Figure 3: Data output for an unsorted array of size 100

```
Original data for ordered100: [137636, 220690, 727172, 827038, 920673, 934768, 950120, 985364, 990370, 997684, 998967, 999762, 999861, 99]

bubbleSort:
Final sorted array: [137636, 220690, 727172, 827038, 920673, 934768, 950120, 985364, 990370, 997684, 998967, 999762, 999861, 99]
Comparisons: 4950
Swaps: 0
Total time: 0.1557 ms

bubbleSort2:
Final sorted array: [137636, 220690, 727172, 827038, 920673, 934768, 950120, 985364, 990370, 997684, 998967, 999762, 999861, 99]
Comparisons: 99
Swaps: 0
Total time: 0.0036 ms

shellSort:
Final sorted array: [137636, 220690, 727172, 827038, 920673, 934768, 950120, 985364, 990370, 997684, 998967, 999762, 999861, 99]
Comparisons: 503
Swaps: 0
Total time: 0.0031 ms
```

Figure 4: Data output for a sorted array of size 100

```
Original data for rand1000: [285734, -76926, -810205, -119061, 403474, -9288, -788972, -787584, -898337, 801119]

bubbleSort:
Final sorted array: [-999152, -993309, -991136, -987397, -986885, -984599, -984388, -983390, -982846, -978769, -972274, -971555]
Comparisons: 499500
Swaps: 246516
Total time: 7.0693 ms

bubbleSort2:
Final sorted array: [-999152, -993309, -991136, -987397, -986885, -984599, -984388, -983390, -982846, -978769, -972274, -971555]
Comparisons: 961038
Swaps: 246516
Total time: 9.6534 ms

shellSort:
Final sorted array: [-999152, -993309, -991136, -987397, -986885, -984599, -984388, -983390, -982846, -978769, -972274, -971555]
Comparisons: 60611
Swaps: 8399
Total time: 0.4959 ms
```

```
Original data for ordered1000: [498393, 600353, 774900, 920497, 951125, 996383, 997056, 999746, 999845, 999964, 999999, 999999, 999999]

bubbleSort:
Final sorted array: [498393, 600353, 774900, 920497, 951125, 996383, 997056, 999746, 999845, 999964, 999999, 999999, 999999, 999999, 999999]
Comparisons: 499500
Swaps: 0
Total time: 0.8881 ms

bubbleSort2:
Final sorted array: [498393, 600353, 774900, 920497, 951125, 996383, 997056, 999746, 999845, 999964, 999999, 999999, 999999, 999999, 999999]
Comparisons: 999
Swaps: 0
Total time: 0.0137 ms

shellSort:
Final sorted array: [498393, 600353, 774900, 920497, 951125, 996383, 997056, 999746, 999845, 999964, 999999, 999999, 999999, 999999, 999999]
Comparisons: 8006
Swaps: 0
Total time: 0.0683 ms
```

Other test cases were also performed to validate the performance of all three sorting methods, including sorting an empty array, a single-element array, a reverse-ordered array, and an array with many duplicates.

```
Empty array:

shellSort:
Final sorted array: []
Comparisons: 0
Swaps: 0
Total time: 0.0000 ms
bubbleSort:
Final sorted array: []
Comparisons: 0
Swaps: 0
Total time: 0.0000 ms
bubbleSort2:
Final sorted array: []
Comparisons: 0
Swaps: 0
Total time: 0.0000 ms
```

Figure 7: Data output for an empty array test case

```

One element:
shellSort:
Final sorted array: [49]
Comparisons: 0
Swaps: 0
Total time: 0.0000 ms
bubbleSort:
Final sorted array: [49]
Comparisons: 0
Swaps: 0
Total time: 0.0000 ms
bubbleSort2:
Final sorted array: [49]
Comparisons: 0
Swaps: 0
Total time: 0.0000 ms

```

Figure 8: Data output for a one element array test case

```

Reverse array:
shellSort:
Final sorted array: [-999, -256, -44, -10, -1, 0, 4, 9, 10, 12, 20, 123, 402, 2301, 2310]
Comparisons: 90
Swaps: 27
Total time: 0.0013 ms
bubbleSort:
Final sorted array: [-999, -256, -44, -10, -1, 0, 4, 9, 10, 12, 20, 123, 402, 2301, 2310]
Comparisons: 105
Swaps: 105
Total time: 0.0045 ms
bubbleSort2:
Final sorted array: [-999, -256, -44, -10, -1, 0, 4, 9, 10, 12, 20, 123, 402, 2301, 2310]
Comparisons: 210
Swaps: 105
Total time: 0.0047 ms

```

Figure 9: Data output for a reverse ordered array test case

```

Array with duplicates:
shellSort:
Final sorted array: [-256, -44, -23, 4, 9, 10, 12, 20, 23, 23, 23, 23, 23, 123, 999]
Comparisons: 142
Swaps: 24
Total time: 0.0016 ms
bubbleSort:
Final sorted array: [-256, -44, -23, 4, 9, 10, 12, 20, 23, 23, 23, 23, 23, 123, 999]
Comparisons: 120
Swaps: 59
Total time: 0.0016 ms
bubbleSort2:
Final sorted array: [-256, -44, -23, 4, 9, 10, 12, 20, 23, 23, 23, 23, 23, 123, 999]
Comparisons: 210
Swaps: 59
Total time: 0.0023 ms

```

Figure 10: Data output for an array with many duplicates test case

Results

The testing results for the number of comparisons, swaps, and total time are summarized below in Table 1. Preliminary observations validate the basic functions of the sorting algorithm, such as 0 swaps for all of the sorted arrays across sorting algorithms. No swaps were required

since each array element was already in the correct order. Additionally, the modification of bubbleSort to bubbleSort2 improved the efficiency by stopping as soon as it realizes that the list is sorted. It makes sense that bubbleSort2 makes 9, 99, and 999 comparisons in the 10, 100, and 1000 element ordered lists because after the first pass is made through the list ($n-1$ comparisons) without swapping any elements, there is no reason to continue. At a more fundamental level, the bubbleSort and bubbleSort2 methods use the same logic which explains why they always make the same number of swaps.

Table 1: Data results for six sets of testing bubbleSort, bubbleSort2, and shellSort

Array Size	Ordered/Random	Sorting Algorithm	Comparisons	Swaps	Total time (ms)
10	Random	bubbleSort	45	25	0.0318
10	Random	bubbleSort2	81	25	0.0145
10	Random	shellSort	69	15	0.0096
10	Ordered	bubbleSort	45	0	0.002
10	Ordered	bubbleSort2	9	0	0.0007
10	Ordered	shellSort	22	0	0.0002
100	Random	bubbleSort	4950	2328	0.2673
100	Random	bubbleSort2	8514	2328	0.3696
100	Random	shellSort	3098	506	0.0412
100	Ordered	bubbleSort	4950	0	0.1557
100	Ordered	bubbleSort2	99	0	0.0036
100	Ordered	shellSort	503	0	0.0031
1000	Random	bubbleSort	499500	246516	7.0693
1000	Random	bubbleSort2	961038	246516	9.6534
1000	Random	shellSort	60611	8399	0.4959
1000	Ordered	bubbleSort	499500	0	0.8881
1000	Ordered	bubbleSort2	999	0	0.0137
1000	Ordered	shellSort	8006	0	0.0683

Further, the total time taken by each sorting algorithm can be analyzed through the graphs below. Figure 11 illustrates the execution time of all three sorting algorithms as the array size increases, with all input arrays being randomly ordered. The y-axis is on a logarithmic scale to highlight differences at both small and large input sizes. For small arrays ($n = 10, 100$), all algorithms complete quickly, though shellSort is consistently the fastest. At $n = 1000$, the performance gap widens significantly. bubbleSort2 takes the longest, followed closely by bubbleSort, reflecting their quadratic time complexity $O(n^2)$ in the unsorted case. This is because the outer for/while loop runs until no swaps happen in a full pass, and in the worst case this

happens up to n times. Inside it, the `for (scan = 0; scan < data.length - 1)` loop runs $n-1$ times per pass. As covered in this chapter, to determine the time complexity these values are multiplied, and $n \times (n - 1) \approx n^2$ for large n . In contrast, `shellSort` maintains much lower execution time, demonstrating superior efficiency on large unsorted datasets because the average efficiency is $O(n \log n)$.

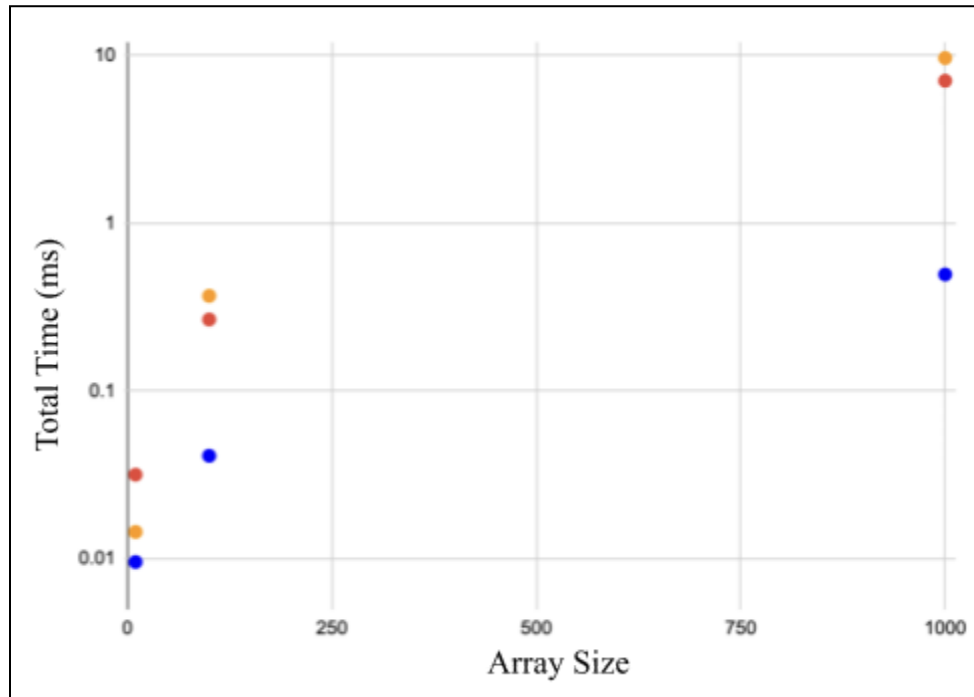


Figure 11: Time (ms) vs. Array Size for *bubbleSort*, *bubbleSort2*, and *shellSort* for unsorted arrays

Figure 12 compares the sorting times for already sorted input arrays. Again, the y-axis uses a logarithmic scale. `bubbleSort2` is fast across the board, particularly at $n = 1000$, where it completes in near-constant time due to its early-exit optimization. `bubbleSort`, lacking this optimization, still iterates fully and therefore takes more time even on sorted data. `shellSort` remains efficient, though slightly slower than `bubbleSort2` at larger sizes due to still performing some comparisons despite the sorted input. The key takeaway is that `bubbleSort2` excels on sorted inputs due to its adaptive nature (best-case $O(n)$, since the loop only executes for one pass), while `bubbleSort` remains inefficient regardless. `shellSort` still performs well, but isn't as fast as `bubbleSort2` in this specific best-case scenario.

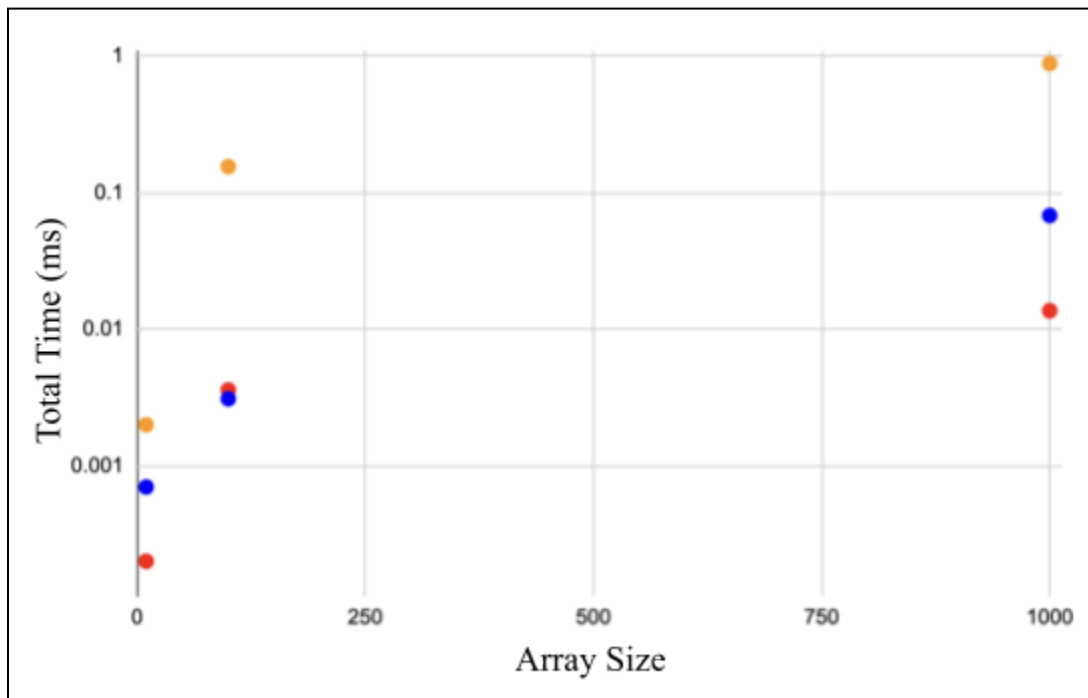


Figure 12: Time (ms) vs. Array Size for *bubbleSort*, *bubbleSort2*, and *shellSort* for sorted arrays